# Software Requirements Specification (SRS)

**Project:** Chat Application (Socket Programming)

**Version:** 1.0

**Authors:** Maansi S / Anand S / Darshan S Govekar / Vismaya G

**Date:** 06-09-2026

**Status:** Draft / For Review

## Revision history

| Version | Date | Author | Change summary | Approval |
|---|---|---|---|---|
| 1.0 | 06-09-2026 | Team of 4 | Initial SRS for real-time WebSocket chat application | |

## Approvals

| Role | Name | Signature / Email | Date |
|---|---|---|---|
| Course Coordinator | Prof. Rajeshwari Adarkatti | | |
| Project Guide | | | |

## Table of Contents

1. Introduction

2. Overall description

3. External interfaces

4. System features (detailed)

5. Non-functional requirements (detailed)

6. Quality attributes & Acceptance tests

7. System models and diagrams

8. Requirements Traceability Matrix (RTM)

# 1. Introduction

### 1.1 Purpose

This document is a Software Requirements Specification (SRS) for a real-time Chat Application developed using socket programming. It defines the functional and non-functional requirements, interfaces, security expectations, and verification criteria for a client-server messaging system that provides instant communication between connected users.

### 1.2 Scope

The system covers user registration/login, user presence, one-to-one and group chat, real-time message delivery using WebSockets, message timestamps, connection management, basic chat history, notifications, and error handling. The project focuses on the application layer and excludes enterprise identity management, telecom services, and production-scale moderation or billing systems.

### 1.3 Audience

Developers, QA engineers, project evaluators, system administrators, security reviewers, and end users involved in the development and evaluation of the chat application.

### 1.4 Definitions

- WebSocket: A persistent, bidirectional communication channel between client and server.
- Node.js: JavaScript runtime used for the server.
- Client: Browser or application interface used by a user.
- Server: Application process that manages WebSocket connections and message routing.
- Room: A logical group of connected users sharing messages.
- Presence: The user's online/offline connection state.
- SRS: Software Requirements Specification.
- SAD: Software Architecture and Design Specification.
- STP: Software Test Plan.

# 2. Overall description

### 2.1 Product perspective

The Chat Application follows a client-server architecture. Clients connect to a Node.js server through WebSockets. The server authenticates users, maintains active connections, routes messages to intended recipients or chat rooms, manages presence information, and returns acknowledgements or errors. A lightweight database or file-based persistence may be used for user and message history data depending on the final implementation.

### 2.2 Major product functions

- User registration and login/logout.
- Maintain user sessions and active WebSocket connections.
- Display online/offline presence.
- Search or select available users for conversation.
- Send and receive one-to-one messages in real time.
- Create and manage group chat rooms.
- Send messages to all members of a group room.
- Attach timestamps to messages.
- Persist and retrieve recent chat history.
- Acknowledge successful message delivery.
- Handle connection loss and reconnection.
- Display unread-message indicators/notifications.

- Validate message content and size.
- Handle server and client errors gracefully.
- Provide basic user profile information such as username and status.

### 2.3 User roles and characteristics

- User: Basic computer/mobile knowledge; expects simple, fast, and understandable messaging.
- Group Creator/Member: Can create or participate in chat rooms and exchange messages with room members.
- Administrator/Developer: Maintains server configuration, reviews logs, and manages deployment/testing.
- QA Tester: Executes functional, security, performance, and reliability test cases.

### 2.4 Operating environment

Client: Modern web browser such as Chrome, Edge, or Firefox. Server: Node.js runtime on Windows/Linux/macOS. Communication: WebSocket over TCP; WSS may be used when TLS is enabled. Development tools may include Visual Studio Code, npm, Git, and a test client/browser.

### 2.5 Constraints

- The application depends on a stable network connection for real-time communication.
- The WebSocket server must manage multiple simultaneous connections without blocking message delivery.
- Message size and room size shall be limited by configurable application settings.
- Authentication credentials and session information must not be exposed through logs or client messages.
- The implementation shall use JavaScript with Node.js and WebSocket-based communication (or Flask-SocketIO only if the team selects the Python implementation).

## 3. External interface requirements

### 3.1 User interfaces

The client UI shall provide login/registration, a user/contact list, chat area, message input, send action, timestamps, connection status, unread indicators, group-chat controls, and clear error/status messages. The UI should be responsive and usable on common desktop and mobile browser sizes.

### 3.2 Hardware interfaces

- No dedicated hardware is required.
- Client devices require a display, keyboard/touch input, and network connectivity.
- The server requires a machine capable of running Node.js and maintaining network socket connections.

### 3.3 Software interfaces

- Node.js runtime and WebSocket server library/API for persistent bidirectional communication.
- Browser WebSocket API for client-server communication.
- Optional database interface for storing users, rooms, and chat history.
- Optional JSON-based REST endpoints for registration/login or metadata where required by the final implementation.

### 3.4 Communications

- WebSocket shall be used for real-time message transport.
- Messages shall use a defined JSON structure containing message type, sender/room identifier, content, timestamp, and message identifier where applicable.
- WSS/TLS shall be used when the application is deployed over an untrusted network.
- The client shall detect closed connections and attempt controlled reconnection.
- The server shall validate all messages received over the socket before processing them.

# 4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow CHAT-F-###.

## 4.1 User Registration and Login

**Description:** The system shall allow a new user to register with a unique username and required credentials, and shall authenticate an existing user before establishing an authenticated chat session.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-001 | The system shall allow a new user to register with a unique username and required credentials, and shall authenticate an existing user before establishing an authenticated chat session. | Functional | High | User | AC: valid registration/login creates an authenticated session. Test: TC-AUTH-01 | Requires user store |

## 4.2 User Logout and Session Management

**Description:** The system shall allow an authenticated user to log out and shall invalidate the associated application session and WebSocket authorization state.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-002 | The system shall allow an authenticated user to log out and shall invalidate the associated application session and WebSocket authorization state. | Functional | High | User | AC: session and socket authorization revoked on logout. Test: TC-AUTH-02 | Requires session manager |

## 4.3 WebSocket Connection Establishment

**Description:** The system shall establish a WebSocket connection for an authenticated client and shall reject unauthorized or malformed connection attempts.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-003 | The system shall establish a WebSocket connection for an authenticated client and shall reject unauthorized or malformed connection attempts. | Functional | High | Architecture | AC: authenticated client reaches connected state. Test: TC-CONN-01 | WebSocket server |

## 4.4 User Presence

**Description:** The system shall maintain online/offline presence based on active connections and shall update relevant users when presence changes.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-004 | The system shall maintain online/offline presence based on active connections and shall update relevant users when presence changes. | Functional | High | User | AC: presence changes after connect/disconnect. Test: TC-PRES-01 | Connection events |

## 4.5 User Search/Selection

**Description:** The system shall allow users to find or select another registered user before starting a one-to-one conversation.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-005 | The system shall allow users to find or select another registered user before starting a one-to-one conversation. | Functional | Medium | User | AC: search returns matching registered users. Test: TC-SEARCH-01 | User directory |

## 4.6 One-to-One Messaging

**Description:** The system shall allow an authenticated user to send a text message to another authenticated user in real time.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-006 | The system shall allow an authenticated user to send a text message to another authenticated user in real time. | Functional | High | User | AC: recipient receives accepted message without manual refresh. Test: TC-MSG-01 | Active connections |

## 4.7 Message Delivery Acknowledgement

**Description:** The system shall return a delivery acknowledgement or error for each accepted message so the sender can determine whether the server processed it.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-007 | The system shall return a delivery acknowledgement or error for each accepted message so the sender can determine whether the server processed it. | Functional | High | User | AC: sender receives ack/error for every send attempt. Test: TC-MSG-02 | Message router |

## 4.8 Group Chat Creation

**Description:** The system shall allow an authorized user to create a chat room and add permitted users as members.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-008 | The system shall allow an authorized user to create a chat room and add permitted users as members. | Functional | High | User | AC: creator can create room and add members. Test: TC-GRP-01 | Room manager |

## 4.9 Group Messaging

**Description:** The system shall broadcast a valid group message to all currently connected members of the selected room, excluding unauthorized users.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-009 | The system shall broadcast a valid group message to all currently connected members of the selected room, excluding unauthorized users. | Functional | High | User | AC: connected authorized members receive the message. Test: TC-GRP-02 | Room membership |

## 4.10 Message Timestamping

**Description:** The system shall associate each accepted message with a server-generated timestamp.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT- | The system shall associate each | Function | Mediu | User | AC: every stored/delivered | Server clock |

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| F-010 | accepted message with a server-generated timestamp. | al | m | | message carries a server timestamp. Test: TC-MSG-03 | |

## 4.11 Chat History

**Description:** The system shall store and retrieve recent messages for an authorized conversation or room, subject to configured retention limits.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-011 | The system shall store and retrieve recent messages for an authorized conversation or room, subject to configured retention limits. | Functional | Medium | User | AC: authorized conversation history is displayed. Test: TC-HIST-01 | Persistence layer |

## 4.12 Unread Message Notification

**Description:** The system shall identify messages received while a conversation is inactive and display an unread indicator to the recipient.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-012 | The system shall identify messages received while a conversation is inactive and display an unread indicator to the recipient. | Functional | Medium | User | AC: unread indicator appears for inactive-conversation messages. Test: TC-NOTIF-01 | Presence/session state |

## 4.13 Message Validation

**Description:** The system shall reject empty, malformed, or over-sized messages and shall return a clear validation error.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-013 | The system shall reject empty, malformed, or over-sized messages and shall return a clear validation error. | Functional | High | Security/QA | AC: invalid payload receives validation error. Test: TC-VAL-01 | Server validation |

## 4.14 Reconnection Handling

**Description:** The client shall detect an unexpected WebSocket disconnection and shall attempt reconnection using a controlled retry mechanism.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-014 | The client shall detect an unexpected WebSocket disconnection and shall attempt reconnection using a controlled retry mechanism. | Functional | High | Reliability | AC: reconnect attempts begin after connection loss. Test: TC-REL-01 | Network dependent |

## 4.15 Error Handling

**Description:** The system shall return standardized error messages for authentication failures, invalid message formats, unavailable recipients, unauthorized rooms, and server-side failures.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-015 | The system shall return standardized error messages for authentication failures, invalid message formats, unavailable recipients, unauthorized rooms, and server-side failures. | Functional | Medium | QA | AC: invalid actions return documented error types. Test: TC-ERR-01 | Error catalogue |

## 4.16 User Profile/Status

**Description:** The system shall allow an authenticated user to view and update basic profile/status information such as display name or status message.

| Req ID | Requirement (shall...) | Type | Priority | Source/ Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| CHAT-F-016 | The system shall allow an authenticated user to view and update basic profile/status information such as display name or status message. | Functional | Medium | User | AC: profile changes persist and display correctly. Test: TC-PROF-01 | User store |

# 5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs: CHAT-NF-###.

| Req ID | Requirement (shall...) | Category | Priority | Acceptance criteria / Measurement |
|---|---|---|---|---|
| CHAT-NF-001 | For normal test load, 90% of accepted chat messages shall receive a server acknowledgement within 1 second. | Performance | High | Test: TC-PERF-01 |
| CHAT-NF-002 | The server shall support at least 100 concurrent WebSocket connections in the project test environment without loss of accepted messages. | Scalability | High | Test: TC-PERF-02 |
| CHAT-NF-003 | If a client connection is interrupted, the client shall detect the closed connection within 10 seconds and initiate controlled reconnection attempts. | Reliability | High | Test: TC-REL-01 |
| CHAT-NF-004 | During a controlled test run, the chat server shall remain available for at least 99% of the scheduled test duration, excluding planned maintenance. | Availability | High | Test: TC-REL-02 |
| CHAT-NF-005 | A new user shall be able to register/login and send a message using the primary UI without needing developer instructions; common actions shall provide clear feedback. | Usability | Medium | Test: TC-UX-01 |
| CHAT-NF-006 | The application shall separate client, WebSocket/server, authentication, message-routing, and persistence responsibilities into identifiable modules. | Maintainability | Medium | Code review + modularity inspection |
| CHAT-NF-007 | The web client shall function correctly on current versions of Chrome, Edge, and Firefox. | Compatibility | Medium | Cross-browser test: TC-COMP-01 |

## 5.1 Security

### 5.1.1 Security Objectives

- Protect user accounts and chat content from unauthorized access or disclosure.
- Ensure that only authenticated and authorized clients can send, receive, or access conversations.
- Preserve the integrity of messages while they are transferred through the WebSocket connection.

### 5.1.2 Security Requirements

| Req ID | Requirement (shall...) | Type | Priority | Acceptance criteria / Test case ref |
|---|---|---|---|---|
| PRJ-SR-001 | All production or non-local network communication shall use WSS/TLS; plaintext WebSocket shall be restricted to controlled local development environments. | Security | High | TC-SEC-01 |
| PRJ-SR-002 | The server shall authenticate the user before granting access to protected WebSocket operations. | Security | High | TC-SEC-02 |
| PRJ-SR-003 | The server shall authorize every message and room operation so that a user cannot send to or read a conversation for which they are not permitted. | Security | High | TC-SEC-03 |
| PRJ-SR-004 | Passwords or authentication secrets shall not be stored or logged in plaintext; credentials shall be protected using an appropriate one-way password hashing mechanism. | Security | High | TC-SEC-04 |
| PRJ-SR-005 | The server shall validate message type, length, identifiers, and room membership to reduce injection, malformed-input, and denial-of-service risks. | Security | High | TC-SEC-05 |
| PRJ-SR-006 | Authentication and message-processing failures shall be logged without recording passwords, session secrets, or private message content. | Security | High | TC-SEC-06 |

## 6. Quality attributes & Acceptance tests

Exit criteria for acceptance: All high-priority functional requirements implemented and verified, no critical defects open, all security requirements tested, and RTM shows coverage for all planned requirements.

**Functional acceptance suite:** registration/login, connection, presence, one-to-one messaging, group messaging, history, validation, logout, and error handling.

**Performance acceptance suite:** message acknowledgement time and concurrent WebSocket connection tests.

**Security acceptance suite:** TLS/WSS, authentication, authorization, credential protection, input validation, and sensitive-data logging checks.

**Usability acceptance suite:** basic navigation, readable feedback, responsive chat interface, and browser compatibility.

# 7. System models and diagrams

## 7.1 UML Use-Case Diagrams

At least 2 UML use-case diagrams should be created for the project. The system (Chat Server) is modeled as the system boundary, not as an actor — only participants external to the system are actors. For this project the only actor is the User; the Chat Server is the system under specification.

**Diagram 1: User Authentication and Session Management**

Actor: User. Use cases: Register, Login, Logout, Establish WebSocket Connection, View/Update Presence.

Relationship: Login <<include>> Establish WebSocket Connection (a successful login always triggers connection setup, per CHAT-F-003).

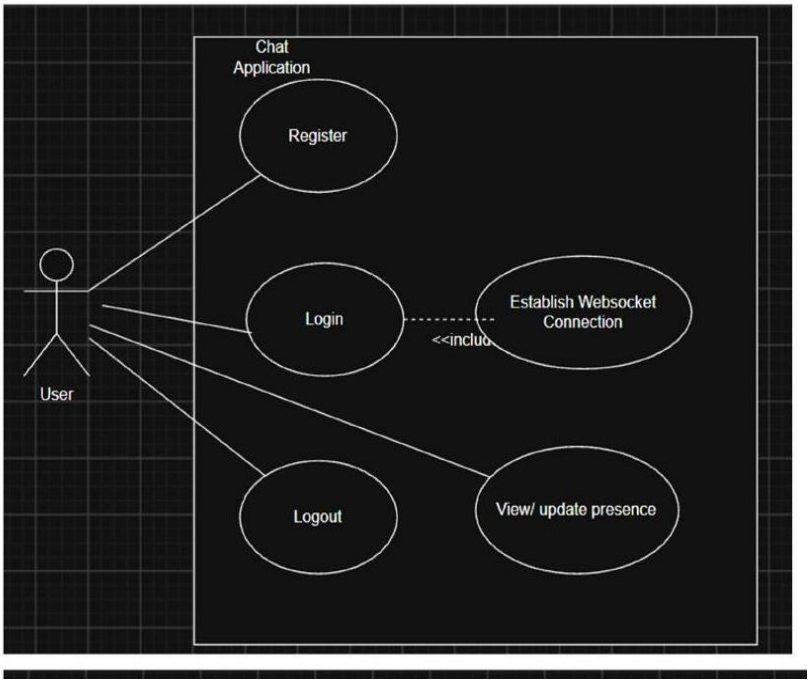
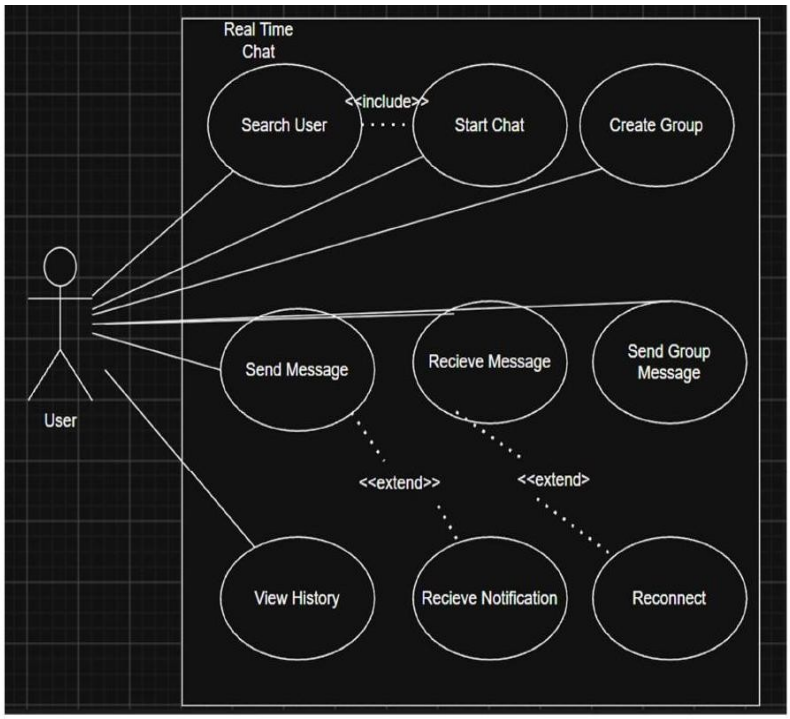



**Diagram 2: Real-Time Chat**

Actor: User. Use cases: Search User, Start Chat, Send Message, Receive Message, Create Group, Send Group Message, View History, Receive Notification, Reconnect.

Relationships: Start Chat <<include>> Search User (a user must be found before a chat can start); Send Message <<extend>> Receive Notification (notification only fires conditionally, per CHAT-F-012); Receive Message <<extend>> Reconnect (reconnection is a conditional/alternate flow triggered on connection loss, per CHAT-F-014).


Real Time
Chat
Search User
<<include>>
Start Chat
Create Group
User
Send Message
Recieve Message
Send Group
Message
<<extend>>
<<extend>
View History
Recieve Notification
Reconnect

# 8. Requirements Traceability Matrix (RTM)

The RTM below covers every functional (CHAT-F-###), non-functional (CHAT-NF-###), and security (PRJ-SR-###) requirement defined in Sections 4 and 5, traced to its section/design reference, implementing module, and test case.

| Req ID | Requirement (short) | Section ref / Design Spec | Module | Test case(s) | Status (N/P/A) | Comments |
|---|---|---|---|---|---|---|
| CHAT-F-001 | Register/Login | 4.1 / DS-Auth-01 | AuthModule / UserStore | TC-AUTH-01 | N | |
| CHAT-F-002 | Logout & session invalidation | 4.2 / DS-Auth-02 | SessionManager | TC-AUTH-02 | N | |
| CHAT-F-003 | Establish WebSocket connection | 4.3 / DS-Conn-01 | WebSocketServer | TC-CONN-01 | N | |
| CHAT-F-004 | Presence tracking | 4.4 / DS-Pres-01 | PresenceModule | TC-PRES-01 | N | |
| CHAT-F-005 | Search/select user | 4.5 / DS-Search-01 | UserDirectory | TC-SEARCH-01 | N | |
| CHAT-F-006 | One-to-one messaging | 4.6 / DS-Msg-01 | MessageRouter | TC-MSG-01 | N | |
| CHAT-F-007 | Delivery acknowledgement | 4.7 / DS-Msg-02 | MessageRouter | TC-MSG-02 | N | |
| CHAT-F-008 | Create group room | 4.8 / DS-Room-01 | RoomManager | TC-GRP-01 | N | |
| CHAT-F-009 | Group broadcast | 4.9 / DS-Room-02 | RoomManager | TC-GRP-02 | N | |
| CHAT-F-010 | Message timestamping | 4.10 / DS-Msg-03 | MessageRouter | TC-MSG-03 | N | |
| CHAT-F-011 | Chat history | 4.11 / DS-Hist-01 | PersistenceModule | TC-HIST-01 | N | |
| CHAT-F-012 | Unread notification | 4.12 / DS-Notif-01 | NotificationModule | TC-NOTIF-01 | N | |
| CHAT-F-013 | Message validation | 4.13 / DS-Val-01 | ValidationModule | TC-VAL-01 | N | |
| CHAT-F-014 | Reconnection handling | 4.14 / DS-Conn-02 | ClientReconnectModule | TC-REL-01 | N | |
| CHAT-F-015 | Standardized errors | 4.15 / DS-Err-01 | ErrorHandler | TC-ERR-01 | N | |
| CHAT-F-016 | Profile/status update | 4.16 / DS-Prof-01 | ProfileModule | TC-PROF-01 | N | |
| CHAT-NF-001 | Ack within 1s (90%) | 5 / DS-Perf-01 | MessageRouter / Server | TC-PERF-01 | N | |
| CHAT-NF-002 | 100 concurrent connections | 5 / DS-Perf-02 | WebSocketServer | TC-PERF-02 | N | |
| CHAT-NF-003 | Disconnect detection <=10s | 5 / DS-Rel-01 | ClientReconnectModule | TC-REL-01 | N | |
| CHAT-NF-004 | 99% test-window availability | 5 / DS-Rel-02 | Server / Infra | TC-REL-02 | N | |
| CHAT-NF-005 | Usable primary UI | 5 / DS-UX-01 | ClientUI | TC-UX-01 | N | |
| CHAT-NF-006 | Modular separation | 5 / DS-Arch-01 | All modules | Code review | N | |
| CHAT-NF-007 | Cross-browser support | 5 / DS-Comp-01 | ClientUI | TC-COMP-01 | N | |
| PRJ-SR-001 | WSS/TLS enforced | 5.1.2 / DS-Sec-01 | TransportSecurity | TC-SEC-01 | N | |
| PRJ-SR-002 | Authenticate before protected ops | 5.1.2 / DS-Sec-02 | AuthModule | TC-SEC-02 | N | |
| PRJ-SR-003 | Authorize message/room ops | 5.1.2 / DS-Sec-03 | AuthorizationModule | TC-SEC-03 | N | |

| Req ID | Requirement (short) | Section ref / Design Spec | Module | Test case(s) | Status (N/P/A) | Comments |
|---|---|---|---|---|---|---|
| PRJ-SR-004 | No plaintext credentials | 5.1.2 / DS-Sec-04 | AuthModule / UserStore | TC-SEC-04 | N | |
| PRJ-SR-005 | Validate message payloads | 5.1.2 / DS-Sec-05 | ValidationModule | TC-SEC-05 | N | |
| PRJ-SR-006 | Safe failure logging | 5.1.2 / DS-Sec-06 | LoggingModule | TC-SEC-06 | N | |

Status legend: N = Not tested, P = Planned/In progress, A = Achieved/Passed.